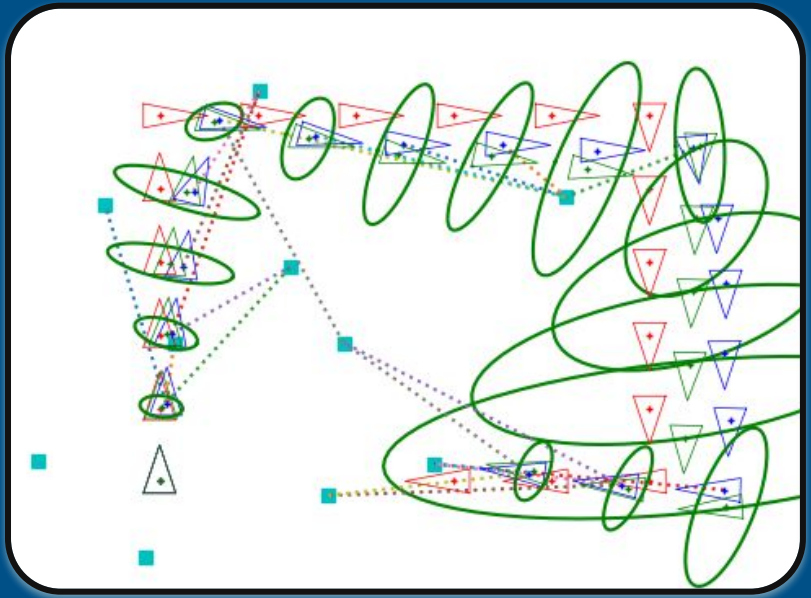
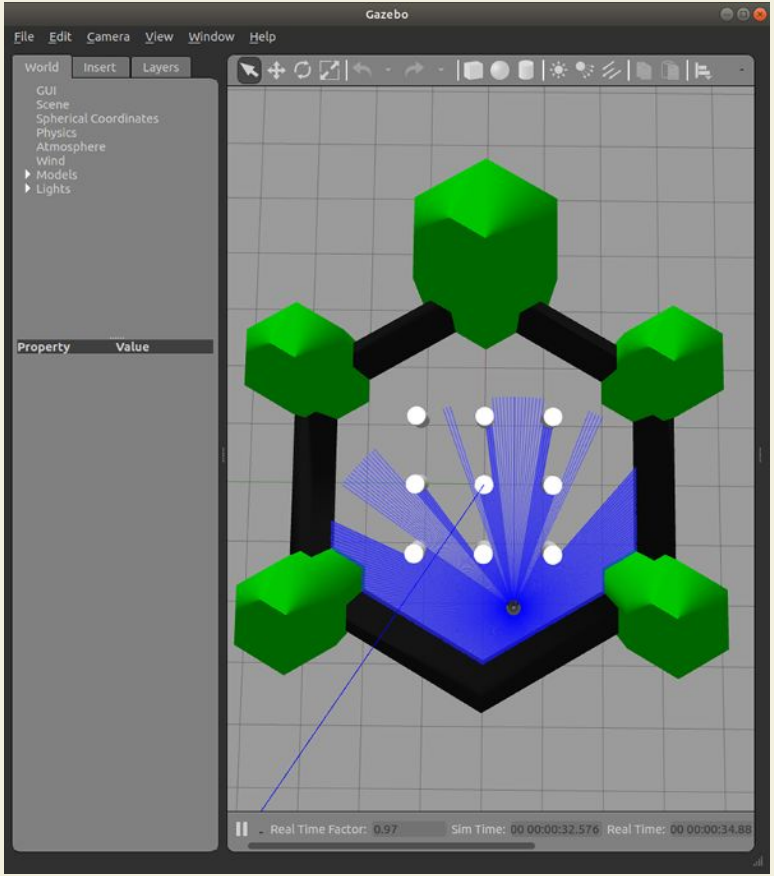
I402: Principios de la Robótica Autónoma

Clases Prácticas
Tutorial 3



Universidad de SanAndrés

Hablemos de la simulación

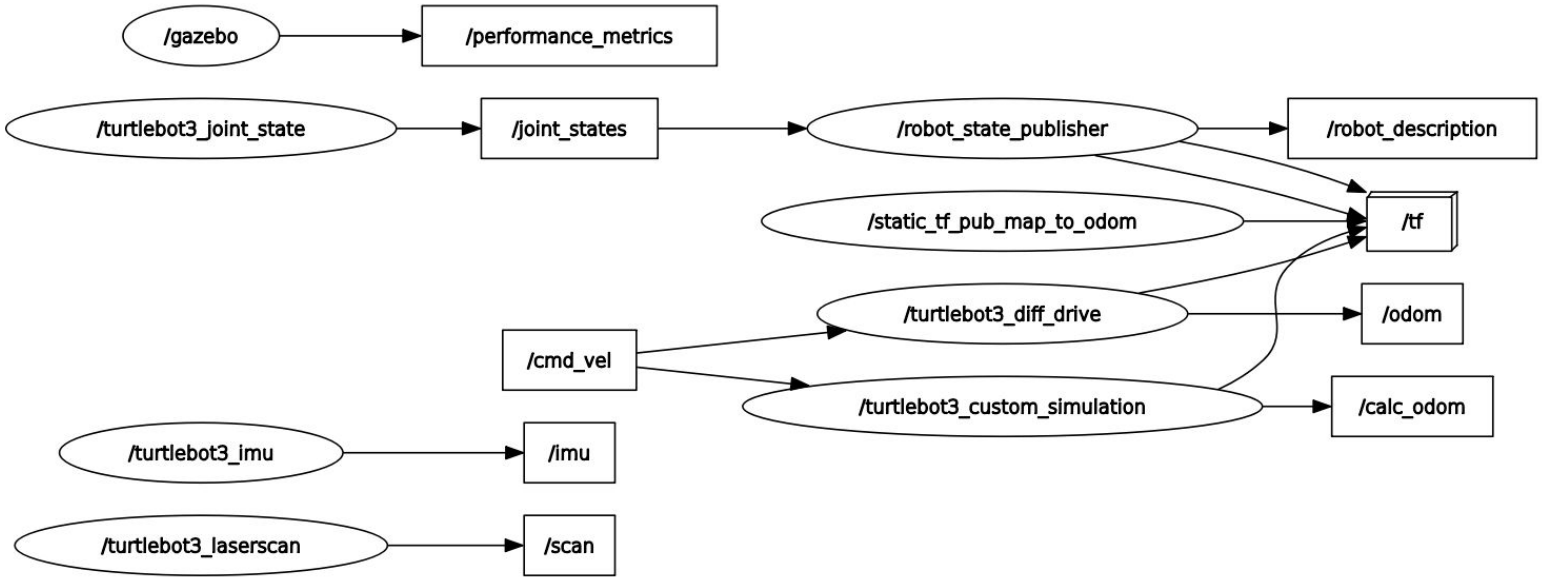


GAZEBO

Hablemos de la simulación

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

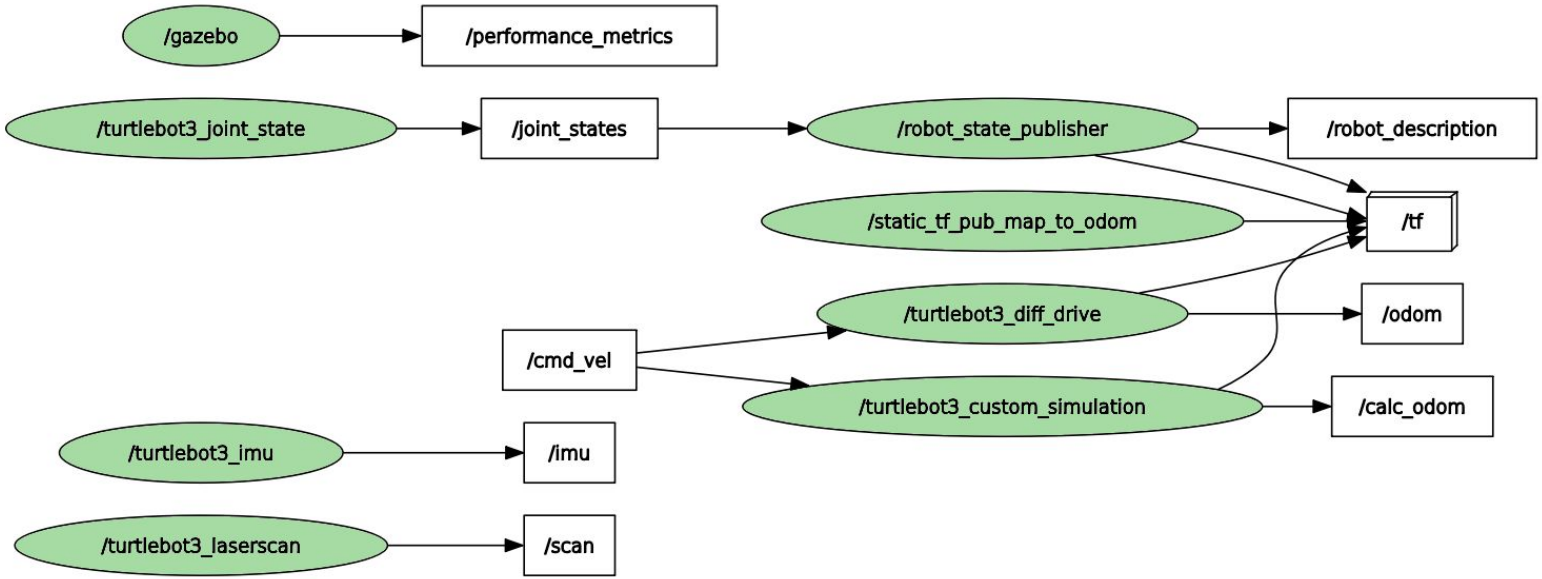
```
rqt_graph
```



Hablemos de la simulación

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

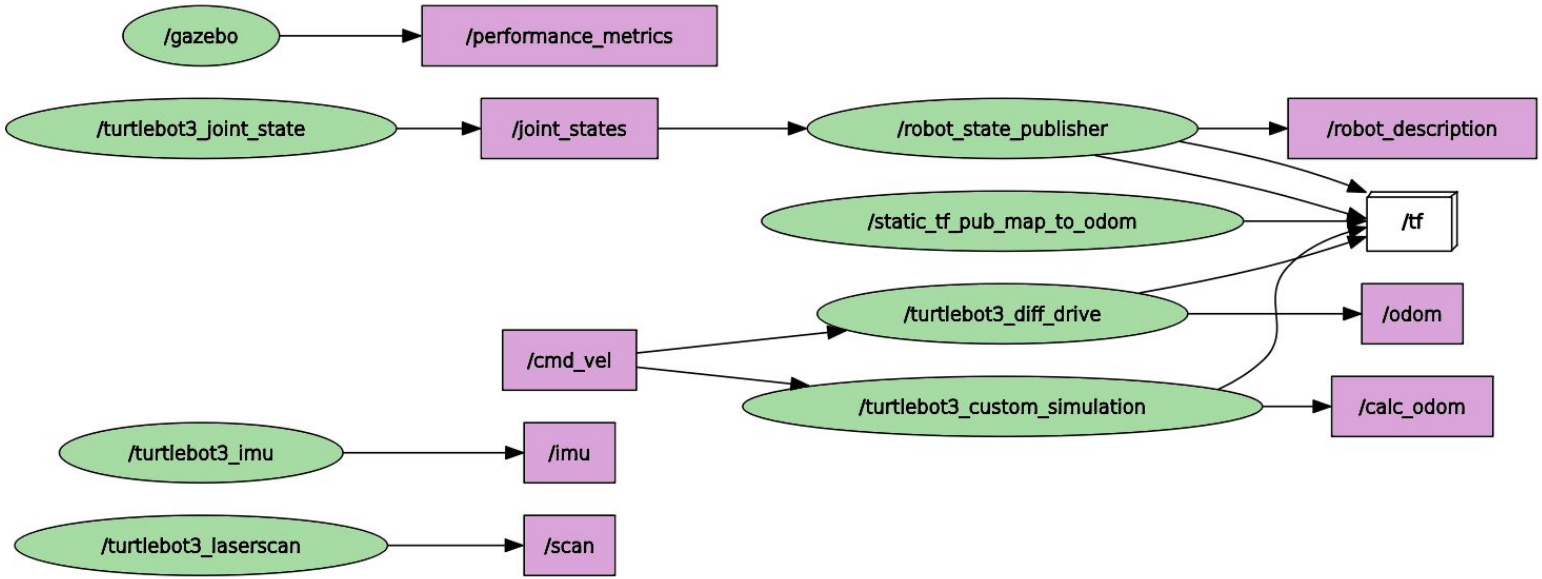
```
rqt_graph
```



Hablemos de la simulación

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

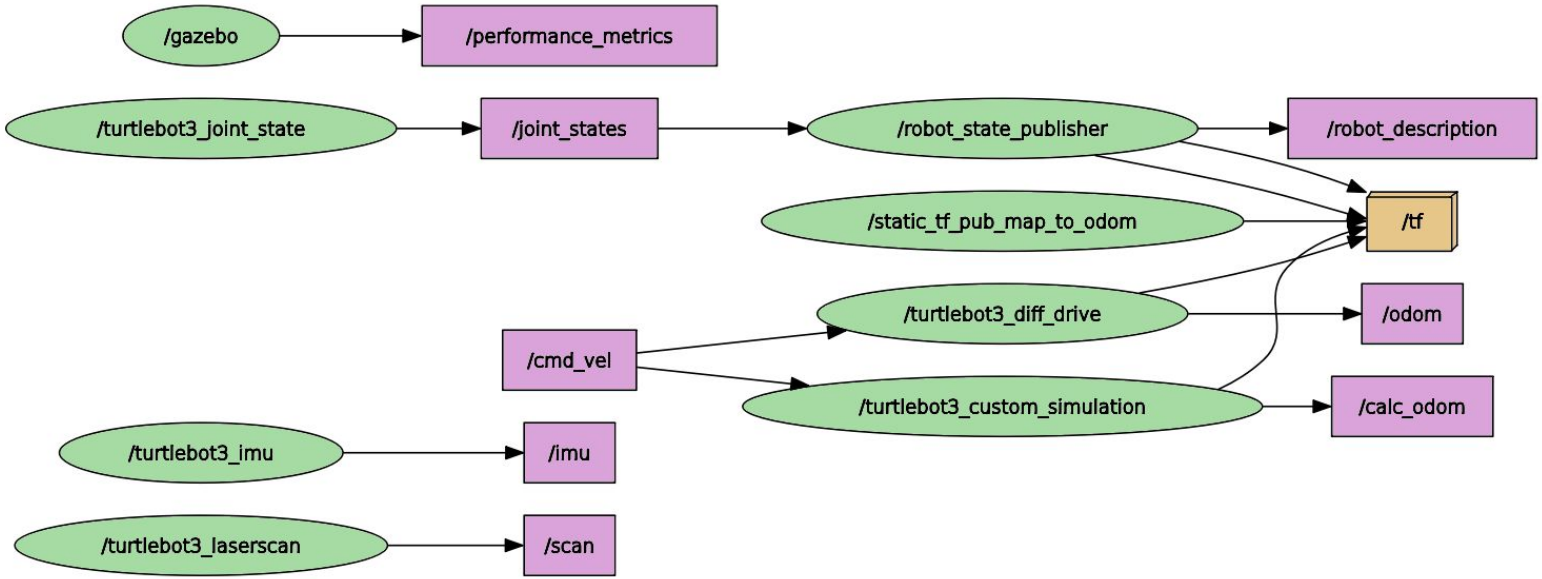
```
rqt_graph
```



Hablemos de la simulación

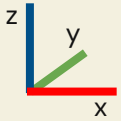
```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

```
rqt_graph
```



Que es la TF?

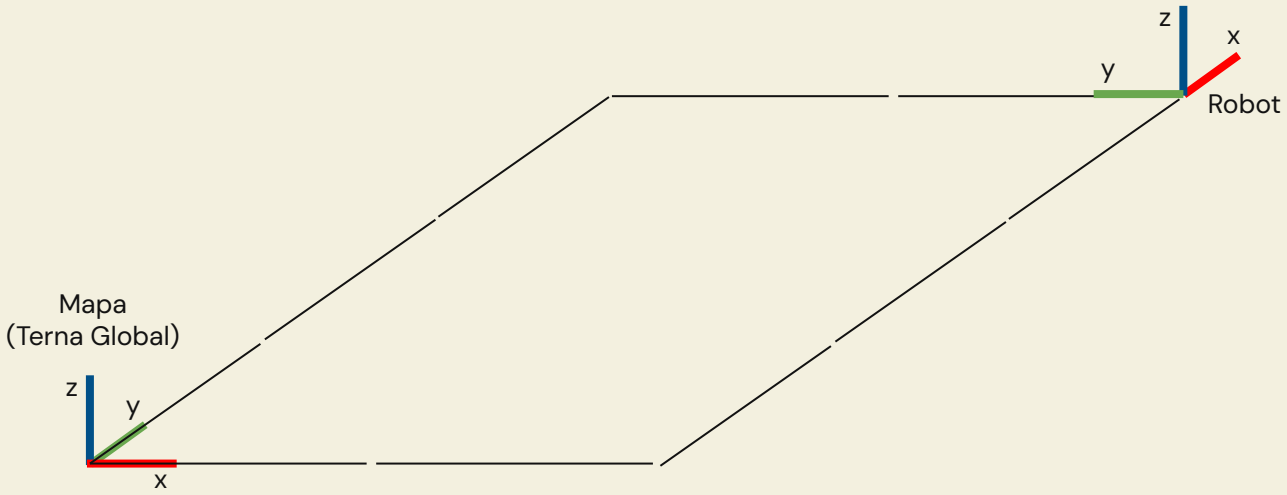
Mapa
(Terna Global)



Que es la TF?

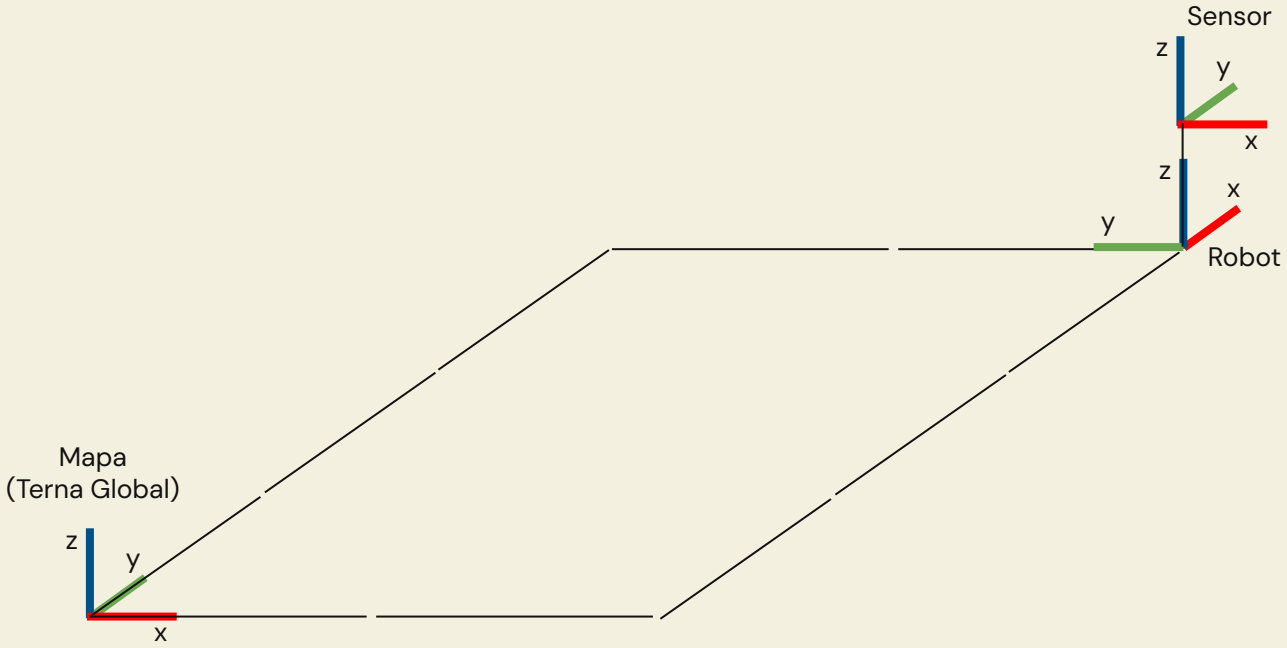
Robot en coordenadas
(2, 3)

Rotado 90 grados respecto al
eje Z

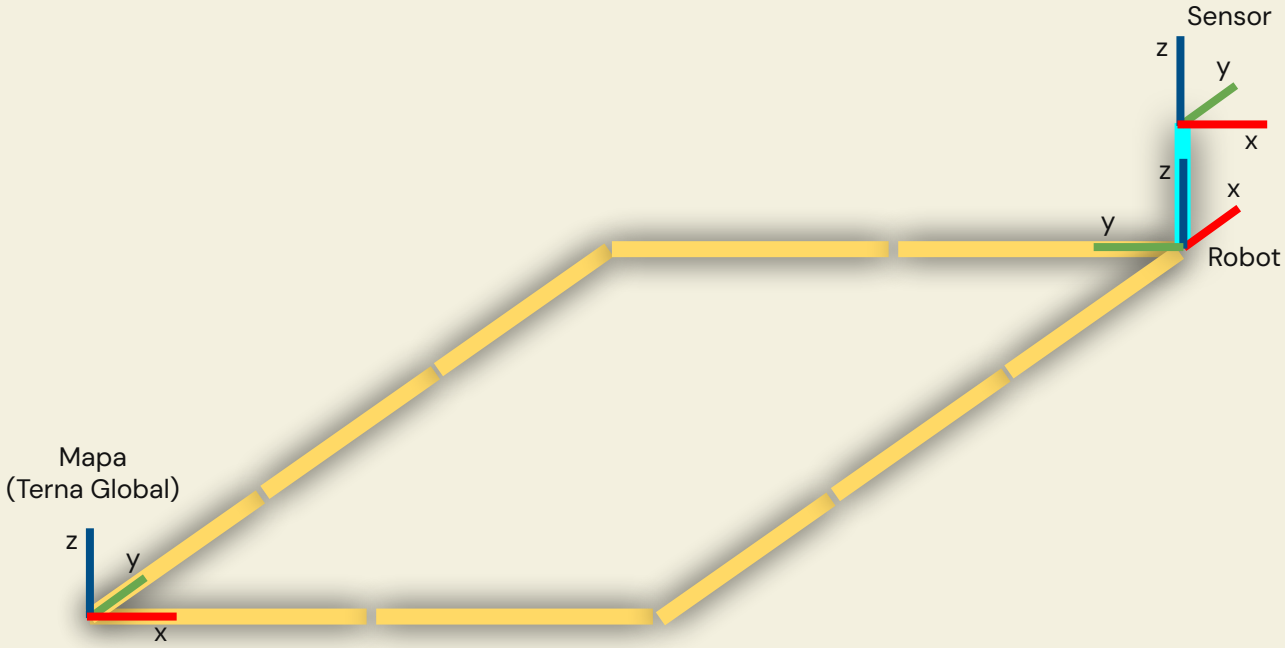


Que es la TF?

Sensor en z + 0.1 según robot y rotado -90 grados en z



Que es la TF?



Que es la TF?

Transformaciones
Dinámicas

Transformaciones
<u>Estáticas</u>

Que es la TF?

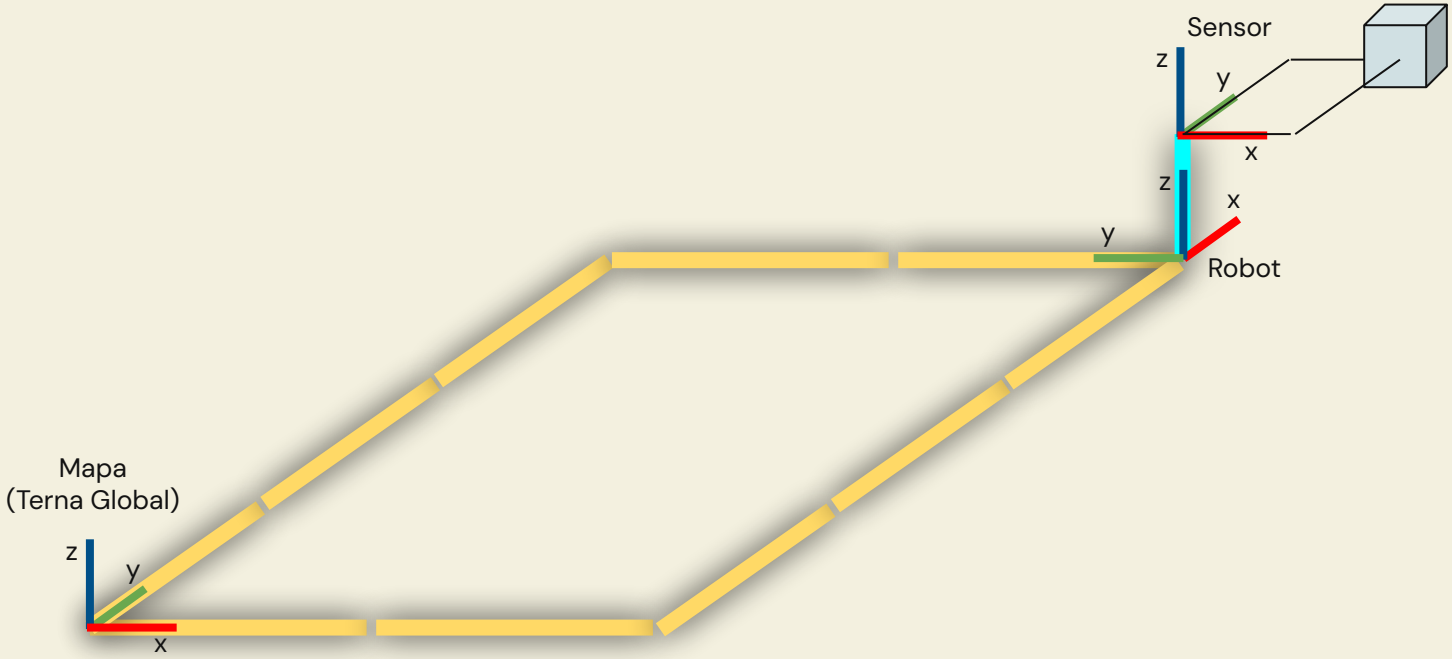
Transformaciones
Dinámicas

- Al principio de la ejecución se definen con el valor inicial
- Son definidas como dinámicas, por lo que permite fácilmente ser modificado en vivo

Transformaciones
Estáticas

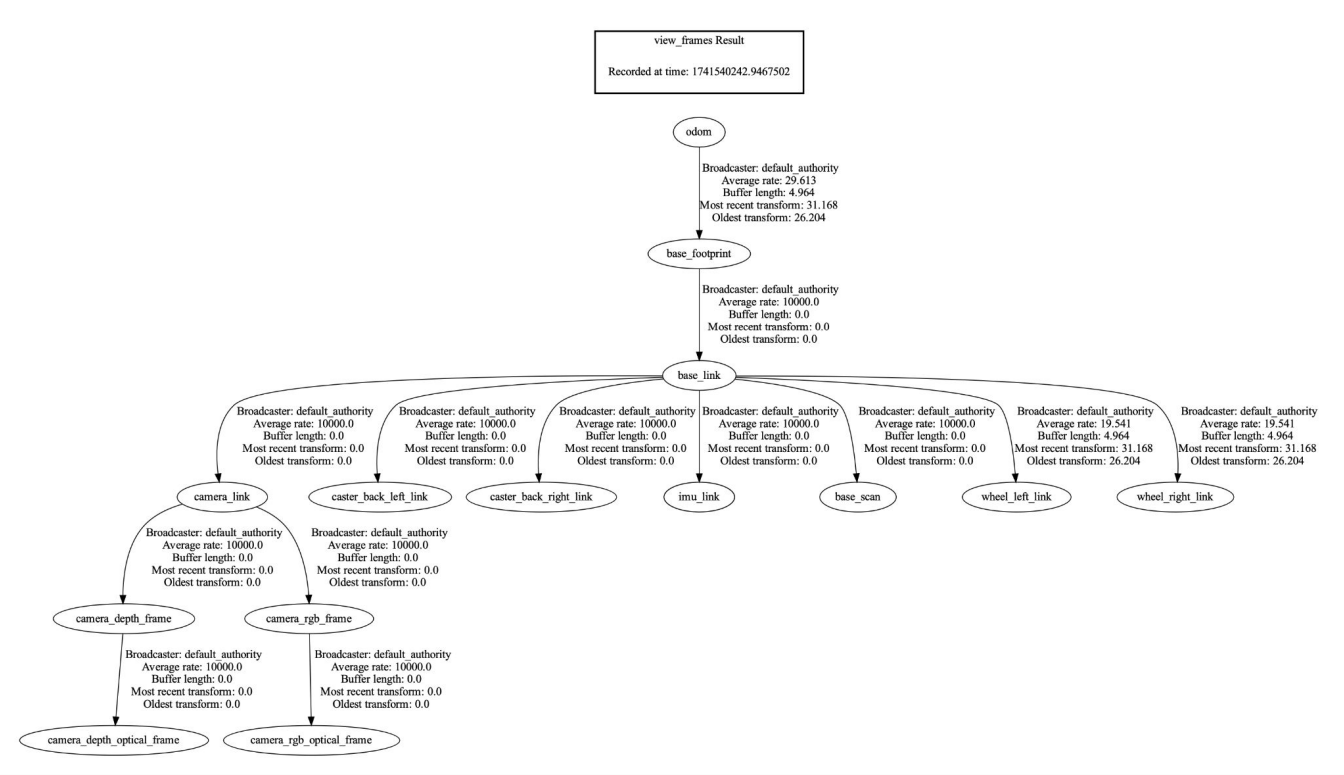
- Se definen una vez, usualmente al principio de la ejecución
- Se definen como estáticas, por lo que no son modificables al menos que se reemplacen

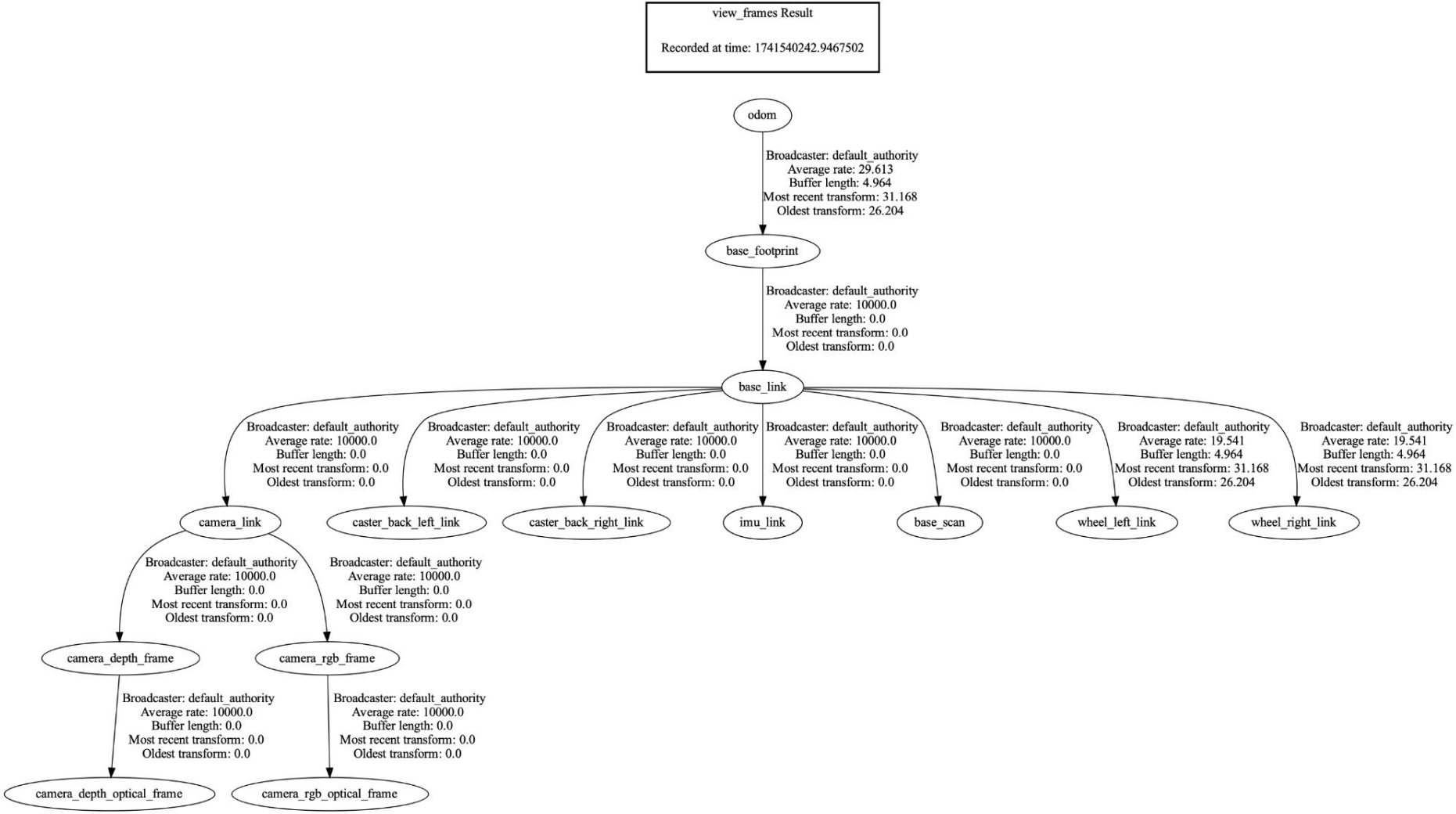
Que es la TF?



Hablemos de la simulación

```
ros2 run tf2_tools view_frames
```




view_frames Result
Recorded at time: 1741540242.9467502
odom
Broadcaster: default_authority
Average rate: 29.613
Buffer length: 4.964
Most recent transform: 31.168
Oldest transform: 26.204
base_footprint
Broadcaster: default_authority
Average rate: 10000.0
Buffer length: 0.0
Most recent transform: 0.0
Oldest transform: 0.0
base_link
Broadcaster: default_authority
Average rate: 10000.0
Buffer length: 0.0
Most recent transform: 0.0
Oldest transform: 0.0
camera_link
Broadcaster: default_authority
Average rate: 10000.0
Buffer length: 0.0
Most recent transform: 0.0
Oldest transform: 0.0
caster_back_left_link
Broadcaster: default_authority
Average rate: 10000.0
Buffer length: 0.0
Most recent transform: 0.0
Oldest transform: 0.0
caster_back_right_link
Broadcaster: default_authority
Average rate: 10000.0
Buffer length: 0.0
Most recent transform: 0.0
Oldest transform: 0.0
imu_link
Broadcaster: default_authority
Average rate: 10000.0
Buffer length: 0.0
Most recent transform: 0.0
Oldest transform: 0.0
base_scan
Broadcaster: default_authority
Average rate: 19.541
Buffer length: 4.964
Most recent transform: 31.168
Oldest transform: 26.204
wheel_left_link
Broadcaster: default_authority
Average rate: 19.541
Buffer length: 4.964
Most recent transform: 31.168
Oldest transform: 26.204
wheel_right_link
camera_depth_frame
Broadcaster: default_authority
Average rate: 10000.0
Buffer length: 0.0
Most recent transform: 0.0
Oldest transform: 0.0
camera_rgb_frame
Broadcaster: default_authority
Average rate: 10000.0
Buffer length: 0.0
Most recent transform: 0.0
Oldest transform: 0.0
camera_depth_optical_frame
Broadcaster: default_authority
Average rate: 10000.0
Buffer length: 0.0
Most recent transform: 0.0
Oldest transform: 0.0
camera_rgb_optical_frame
Broadcaster: default_authority
Average rate: 10000.0
Buffer length: 0.0
Most recent transform: 0.0
Oldest transform: 0.0

Hablemos de la simulación

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

```
ros2 topic ...
```

Hablemos de la simulación

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

```
ros2 topic list
```

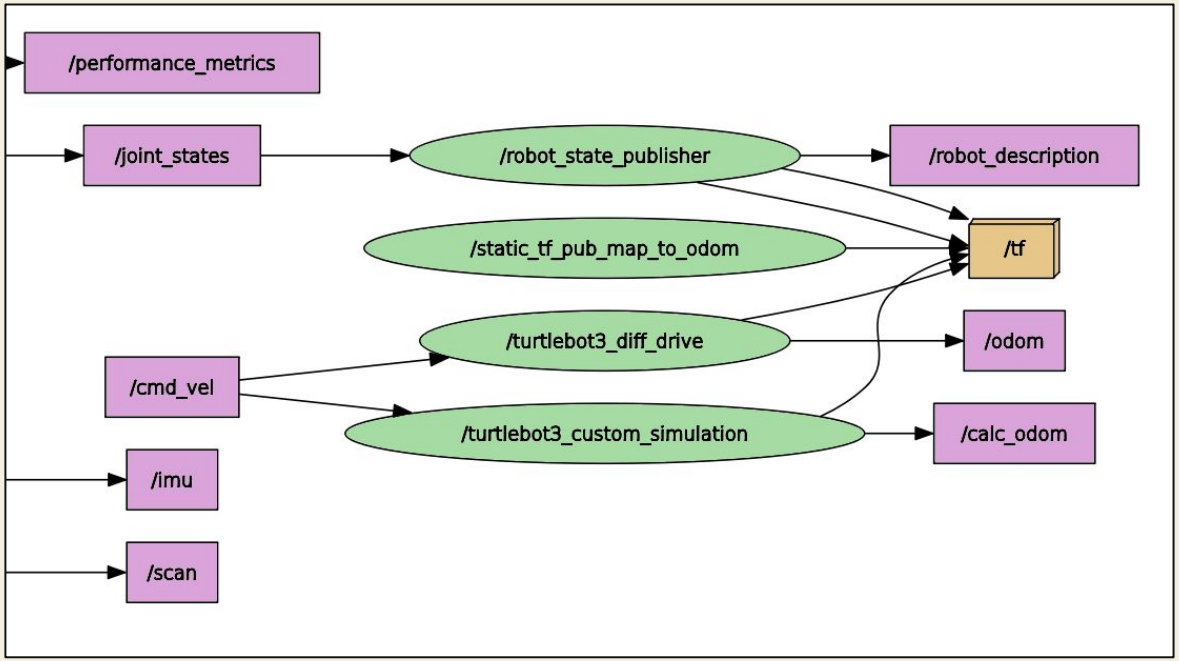
/calc_odom
/clock
/cmd_vel
/imu
/joint_states
/odom
/parameter_events
/performance_metrics
/robot_description
/rosout
/scan
/tf
/tf_static

Hablemos de la simulación


```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

```
ros2 topic list
```

/calc_odom
/clock debug
/cmd_vel
/imu
/joint_states
/odom
/parameter_events debug
/performance_metrics debug
/robot_description
/rosout debug
/scan
/tf
/tf_static



Hablemos de la simulación

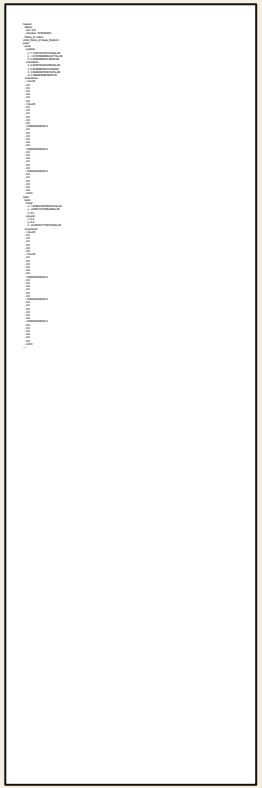
```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

```
ros2 topic echo “topic”
```

Hablemos de la simulación

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

```
ros2 topic echo calc_odom
```



Veámoslo en la terminal mejor...

Hablemos de la simulación

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

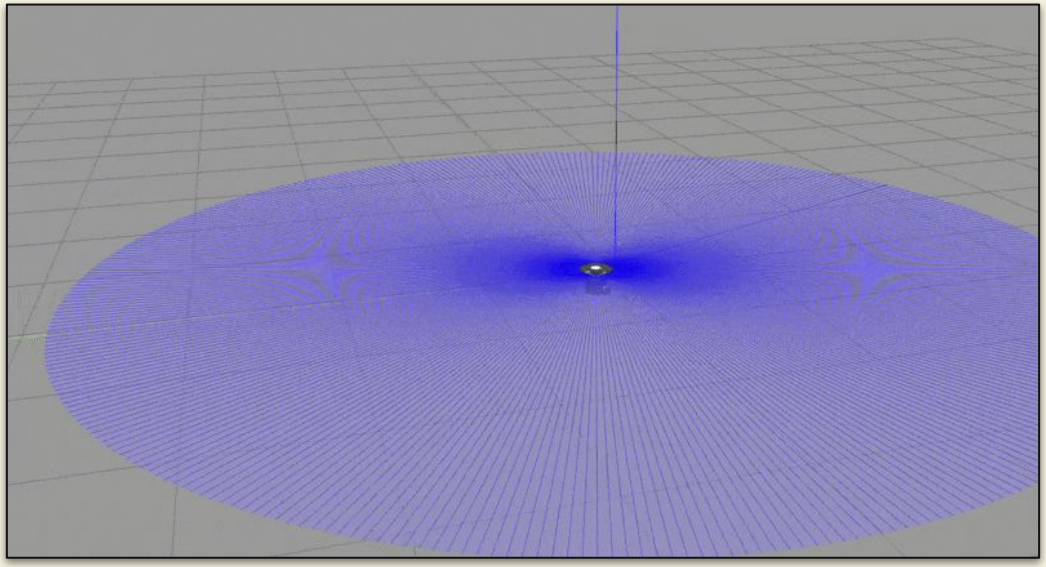
```
ros2 topic echo joint_states
```

```
header:
  stamp:
    sec: 477
    nanosec: 191000000
  frame_id: ''
name:
- wheel_left_joint
- wheel_right_joint
position:
- 0.0008385192819595488
- 0.0008969842889028357
velocity:
- 2.6076174873535234e-07
- 3.429416156439276e-07
effort: []
```

Hablemos de la simulación

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{ linear: { x: 0.2, y: 0.0, z: 0.0 },
angular: { x: 0.0, y: 0.0, z: 0.5 }}"
```



Hablemos de la simulación

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

```
ros2 run turtlebot3_teleop teleop_keyboard
```

Hablemos de la simulación

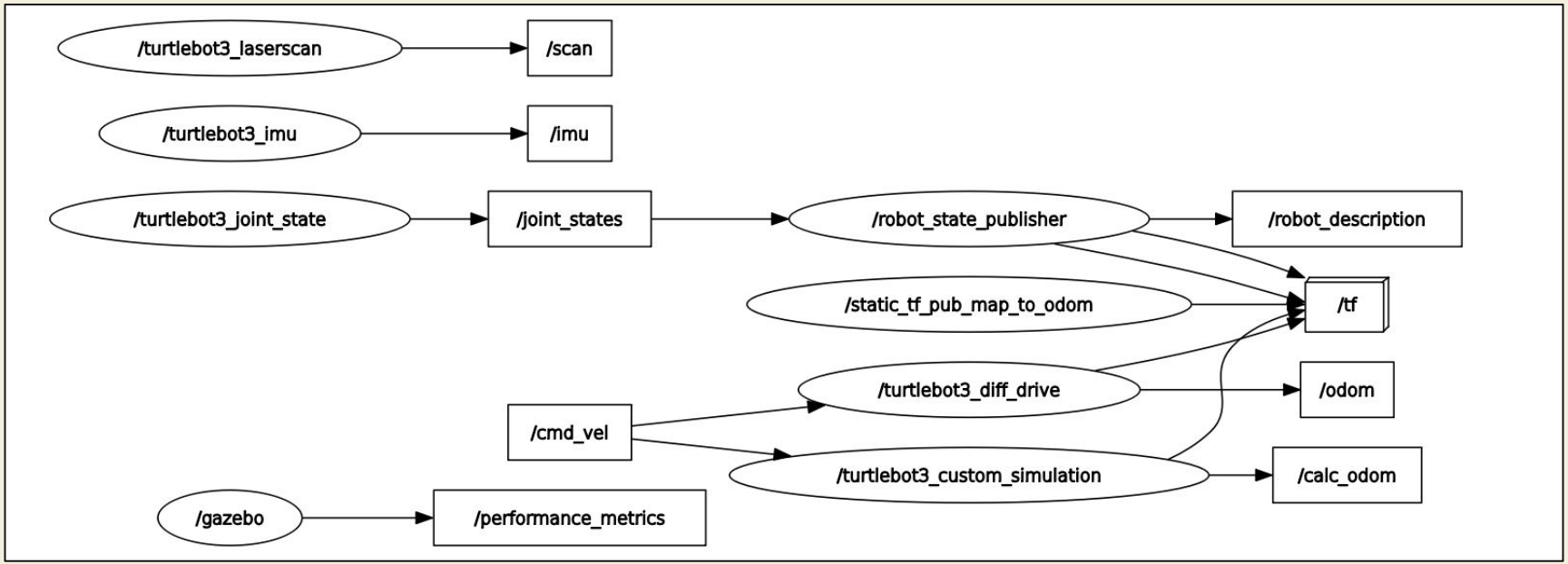
```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

```
ros2 run turtlebot3_teleop teleop_keyboard
```

```
Control Your TurtleBot3!
---------------------------
Moving around:
        w
   a    s    d
        x

w/x : increase/decrease linear velocity (Burger : ~ 0.22, Waffle and Waffle Pi : ~ 0.26)
a/d : increase/decrease angular velocity (Burger : ~ 2.84, Waffle and Waffle Pi : ~ 1.82)

space key, s : force stop

CTRL-C to quit
```

Hablemos de la simulación

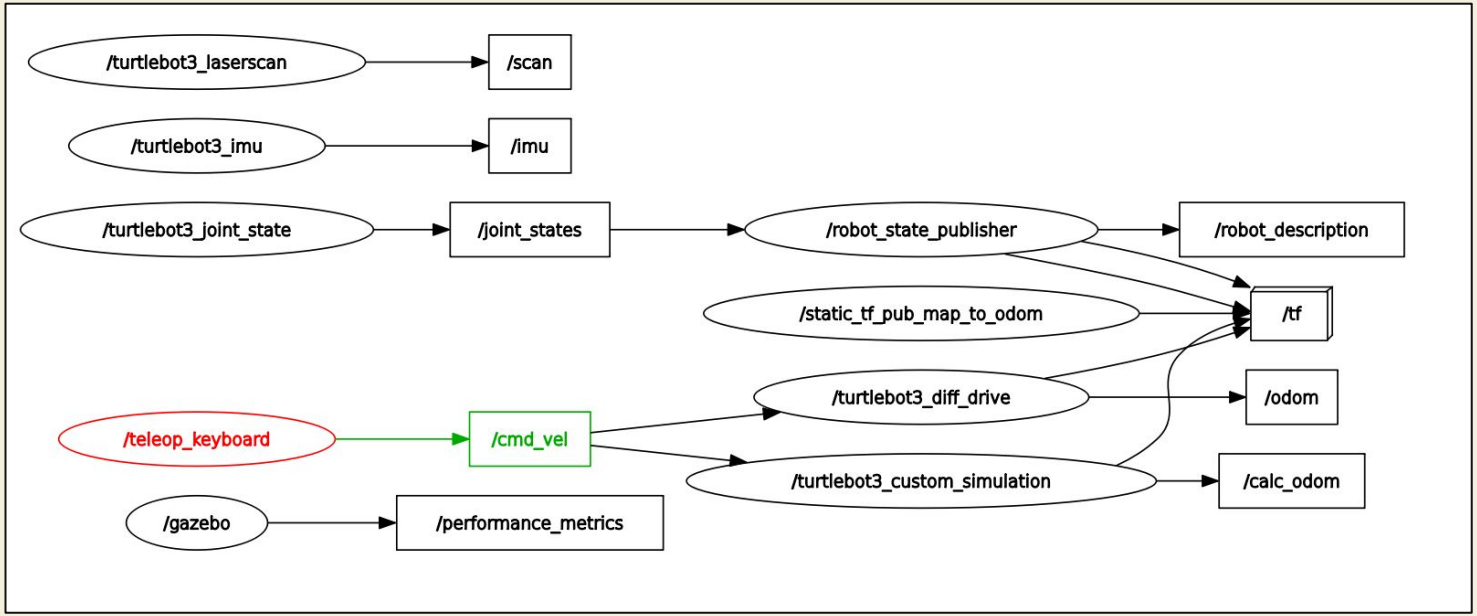
```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```



Hablemos de la simulación

```
ros2 launch turtlebot3_custom_simulation custom_simulation.launch.py
```

```
ros2 run turtlebot3_teleop teleop_keyboard
```



Quiero que el robot:

- se mueva a 0.2 m/s en x
- Tenga una velocidad angular sobre el eje z que:
  - Sea aleatoria entre -pi/2 y pi/2
  - Cambie cada 4 segundos

Nuestro paquete

1. Creamos el workspace y dentro creamos la carpeta src
2. Creamos el paquete dentro de src

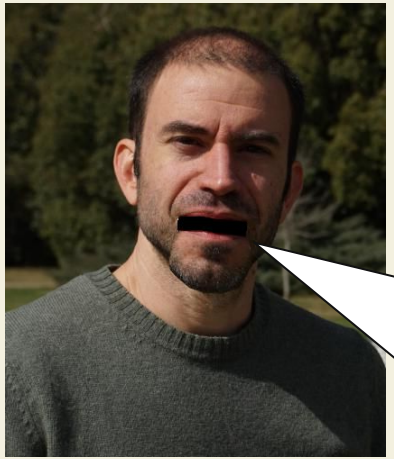
```
ros2 pkg create --build-type ament_python --license Apache-2.0 <pkg_name>
```

3. Creamos y agregamos el código de nuestro nodo
4. Modificamos package.xml y setup.py
5. Compilamos todo desde la carpeta del workspace

```
colcon build --symlink-install
```

6. Hacemos “source install/setup.bash” (ojo si no tienen bash)
7. Corremos el nodo

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import random
from geometry_msgs.msg import Twist
import numpy as np

class Commander(Node):
    def __init__(self):
        super().__init__('commander')
        self.publisher = self.create_publisher(Twist, '/cmd_vel', 10)
        self.timer = self.create_timer(0.05, self.timer_callback)
        self.counter = 0
        self.target = int(4/0.05)
        self.w = 0.0
        self.get_logger().info("Randomly sending rotation commands to the robot every 4 seconds")

    def timer_callback(self):
        if self.counter == self.target:
            self.counter = 0
            self.w = random.uniform(-1, 1) * np.pi/2
            self.get_logger().info(f"Setting the angular velocity to {self.w}")
        else:
            self.counter += 1
        msg = Twist()
        msg.linear.x = 0.2
        msg.angular.z = self.w
        self.publisher.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = Commander()
    try:
        rclpy.spin(node)  # Keep ROS 2 running
    except KeyboardInterrupt:
        print("\nShutting down...")
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```



Bien, ahora quiero que al finalizar la ejecución me hagan un gráfico del camino que hizo el robot.

Toma/Visualización de datos

1. ROS bag
2. Rviz2 (La clase que viene)
3. Guardamos los datos en un archivo
4. Usamos matplotlib

```python
import rclpy
from rclpy.node import Node
import os
from pathlib import Path
import atexit
from nav_msgs.msg import Odometry

class Recorder(Node):

    def __init__(self):
        super().__init__('recorder')

        self.data_sub = self.create_subscription(
            Odometry,
            '/odom',
            self.odom_callback,
            10)
        self.data_sub  # prevent unused variable warning

        self.coordinates_x = []
        self.coordinates_y = []

        self.dataLoc = r"turtlebot3_ws/src/commander"
        self.datafile = os.path.join(str(Path.home()), self.dataLoc, 'coord.csv')

    def odom_callback(self, msg):
        self.coordinates_x.append(msg.pose.pose.position.x)
        self.coordinates_y.append(msg.pose.pose.position.y)

    def save(self):
        with open(self.datafile,'w') as file:
            header = 'x,y\n'
            file.write(header)
            for coordx, coordy in zip(self.coordinates_x, self.coordinates_y):
                msg = f'{coordx},{coordy}\n'
                file.write(msg)
```

```python
def main(args=None):
    rclpy.init(args=args)

    minimal_subscriber = Recorder()
    atexit.register(minimal_subscriber.save)
    rclpy.spin(minimal_subscriber)

    minimal_subscriber.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```